Sistema web de inventario para D'PAO Cookies: Avance de Proyecto Final 2

Aplicaciones web transaccionales y patrones de diseño empresariales

Anderson Enrique Junior Casas Gonzales (U22244177)

Carrera de Ingeniería de Sistemas e Informática, Facultad de Ingeniería, Universidad Tecnológica del Perú

100000SI54: Desarrollo Web Integrado, Sección 20061

Mtro. César Augusto Cárdenas Latorre

Ciclo 2026-2. Avance de Proyecto Final 2 (APF2, 30 %)

8 de octubre de 2026

Índice

Introducción y alcance de la Unidad 2....3
Objetivo del avance....3
Requerimientos de la Unidad 2 y resultado....3
Punto de partida y límites....4
Arquitectura de software y patrones de diseño....4
Patrón MVC + DAO con BaseServlet y servlets modulares....5
Capa DTO: justificación y estructura....6
Patrón Façade: InventarioFacade....7
Diagrama conceptual de capas....9
Módulo transaccional de Kardex....10
Atomicidad transaccional JDBC....10
Bloqueo pesimista y trazabilidad de saldos....12
Saldo no negativo y ajustes físicos....13
Capa de presentación con JSTL 1.2 y EL....14
Erradicación total de scriptlets....14
JSTL, EL y formateo regional....15
Verificación, aseguramiento de calidad y pruebas....17
Resumen de las 41 pruebas automatizadas....17
Concurrencia multihilo y rollback forzado....19
Pruebas HTTP de humo....19
Evidencias del despliegue y ejecución....20
Pruebas y control de versiones....25
Matriz de cumplimiento de la rúbrica....26
Pendientes de evolución....27
Referencias....28

Sistema web de inventario para D'PAO Cookies: Avance de Proyecto Final 2

Introducción y alcance de la Unidad 2

D'PAO Cookies elabora y comercializa galletas artesanales. En el APF1 se entregó un prototipo web Java (Servlets, JSP y JDBC) que centraliza insumos y productos y registra las variaciones de stock en MySQL. La Unidad 2 exige que esa base evolucione hacia una aplicación transaccional organizada con patrones de diseño empresariales, con reglas de negocio explícitas y pruebas automatizadas que demuestren la atomicidad de las operaciones. El código fuente se encuentra en la rama apf2 del repositorio del proyecto (Casas Gonzales, 2026).

Objetivo del avance

Reorganizar la aplicación en capas desacopladas —controladores, fachada de negocio, objetos de transferencia y acceso a datos—, garantizar que todo movimiento de Kardex sea atómico y trazable, eliminar el código Java de las vistas y verificar el resultado con pruebas unitarias, de integración y HTTP.

Requerimientos de la Unidad 2 y resultado

La Tabla 1 relaciona cada requerimiento técnico del enunciado con el resultado comprobado en esta entrega.

Tabla 1

Requerimientos técnicos del APF2 y estado comprobado

Requerimiento	Resultado comprobado
MVC + DAO	Cuatro servlets modulares sobre BaseServlet; ningún servlet importa un DAO. Los DAO devuelven entidades tipadas en lugar de mapas genéricos.
DTO	Seis clases en pe.com.dpaocookies.dto; las vistas reciben exclusivamente DTO.
Façade	InventarioFacade es el único punto de entrada al negocio: valida, orquesta, mapea y traduce excepciones.
Kardex transaccional	setAutoCommit(false), SELECT … FOR UPDATE, commit() y rollback(), saldos anterior y posterior, rechazo de saldos negativos y ajustes por conteo.
JSTL y EL	Cero scriptlets; <scripting-invalid>true</scripting-invalid> lo impone en el contenedor.
Calidad	41 pruebas JUnit aprobadas (0 omitidas) contra MySQL real, BUILD SUCCESS y 55 solicitudes HTTP verificadas.

Punto de partida y límites

Se conservan el esquema relacional sql/schema.sql del APF1, la autenticación con cuenta externa, la protección CSRF y el patrón Post/Redirect/Get. Siguen fuera del alcance las cuentas múltiples y roles, el responsable individual por movimiento, las recetas con consumo automático, los lotes y la caducidad, la paginación, los comprobantes de venta y el despliegue público con HTTPS.

Arquitectura de software y patrones de diseño

El código se organiza en paquetes por responsabilidad bajo pe.com.dpaocookies, siguiendo el catálogo de patrones para aplicaciones Java empresariales (Alur et al., 2003). Cada capa depende solo de la inmediatamente inferior, y los objetos que cruzan las fronteras son DTO (hacia la vista) o entidades (hacia la persistencia), como resume la Tabla 2.

Tabla 2

Paquetes y responsabilidades

Paquete	Clases	Responsabilidad
controller	BaseServlet, TableroServlet, InsumoServlet, ProductoServlet, KardexServlet, AccesoServlet, SeguridadFilter	Recibir HTTP, convertir parámetros, delegar en la fachada y elegir la vista.
facade	InventarioFacade, MapeadorDTO	Reglas de negocio, orquestación de DAO, mapeo entidad → DTO y traducción de errores.
dto	InsumoDTO, ProductoDTO, CategoriaDTO, MovimientoDTO, SolicitudMovimientoDTO, ResumenTableroDTO	Transferir datos entre capas sin exponer entidades.
dao	InsumoDAO, ProductoDAO, CategoriaDAO, KardexDAO	SQL parametrizado y transacciones JDBC.
model	Insumo, Producto, Categoria, MovimientoKardex, TipoMovimiento, ClaseItem	Entidades de tabla y enumeraciones de dominio.
exception	AplicacionException, NegocioException, StockInsuficienteException, ServicioException	Errores con mensaje apto para el usuario.
util	ConexionBD, ProveedorConexion, Validacion	Conexión externalizada y reglas de formato compartidas.

Patrón MVC + DAO con BaseServlet y servlets modulares

El PanelServlet del APF1 atendía tres rutas con lógica mezclada. Se reemplazó por un servlet por módulo, todos derivados de BaseServlet, que concentra las tareas comunes: conversión de parámetros a BigDecimal, int o LocalDate con mensajes de error comprensibles, mensajes de un solo uso y redirección GET tras cada POST. El patrón Data Access Object aísla el SQL en clases dedicadas (Alur et al., 2003), y una búsqueda en el paquete controller confirma que ninguna clase importa pe.com.dpaocookies.dao. La Tabla 3 muestra qué operación de la fachada invoca cada ruta y la Figura 1, el controlador del Kardex.

Tabla 3

Rutas, controladores y operaciones de la fachada

Ruta	Servlet	GET → fachada	POST → fachada
/tablero	TableroServlet	obtenerResumen()	—
/insumos	InsumoServlet	listarInsumos(), listarCategorias()	registrarInsumo(), ajustarStockInsumo()
/productos	ProductoServlet	listarProductos(), listarCategorias()	guardarProducto()
/kardex	KardexServlet	listarInsumos(), listarProductos(), consultarKardex()	registrarMovimiento()
/acceso, /salir	AccesoServlet	Formulario de acceso	Credenciales externas; invalidación de sesión

Figura 1

Método doPost de KardexServlet: el controlador arma el DTO y delega en la fachada

```
try {
   String[] item = texto(request, "item").split(":", 2);   // "PRODUCTO:3"
   SolicitudMovimientoDTO solicitud = new SolicitudMovimientoDTO(
         item[0], parsearId(item[1]), texto(request, "tipo"),
         decimal(request, "cantidad", "La cantidad no es válida."),
         texto(request, "motivo"));
   Optional<MovimientoDTO> movimiento = facade.registrarMovimiento(solicitud);
   exito(request, movimiento.map(m -> "Movimiento registrado: " + ...)
         .orElse("El stock contado coincide con el actual; no se registró ningún
movimiento."));
} catch (AplicacionException exception) {
   fallo(request, exception.getMessage());
}
redirigir(request, response);   // Post/Redirect/Get
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/controller/KardexServlet.java.

Capa DTO: justificación y estructura

Un Data Transfer Object agrupa los datos que viajan entre capas en un único objeto serializable, sin lógica de persistencia (Fowler, 2002). Antes, ProductoDAO y KardexDAO devolvían List<Map<String,Object>> y las JSP dependían de nombres de columna como p.id_producto; un cambio de esquema rompía la vista sin error de compilación. Ahora cada

DTO es un JavaBean Serializable con getters que EL resuelve por convención (${insumo.stockActual}) e incorpora datos derivados que la entidad no guarda (Tabla 4).

Tabla 4

Objetos de transferencia de datos

DTO	Dirección	Campos relevantes	Datos derivados
InsumoDTO	Formulario ↔ vista	código, nombre, unidad, stock actual y mínimo, costo, categoría	nombreCategoria, bajoStock
ProductoDTO	Formulario ↔ vista	código, nombre, precio de venta, stock, categoría	nombreCategoria
CategoriaDTO	Vista	idCategoria, nombre	—
SolicitudMovimientoDTO	Formulario → fachada	claseItem, idItem, tipoMovimiento, cantidad, motivo	En AJUSTE, cantidad = stock contado
MovimientoDTO	Fachada → vista	ítem, tipo, cantidad con signo, stock anterior y posterior, fecha	isEntrada() para el estilo de la fila
ResumenTableroDTO	Fachada → vista	totales, alertas, cinco últimos movimientos	cantidadAlertas; listas inmutables

La conversión vive en MapeadorDTO, una clase de paquete dentro de facade; los DTO no conocen las entidades y las entidades no conocen los DTO, de modo que ninguna entidad sale de la fachada.

Patrón Façade: InventarioFacade

Una fachada proporciona una interfaz unificada y de más alto nivel sobre un conjunto de interfaces de un subsistema (Gamma et al., 1994). InventarioFacade ofrece esa interfaz sobre cuatro DAO: recibe DTO, valida formato y reglas (textos obligatorios, decimales con dos cifras, identificadores positivos, rango de fechas y cantidad mayor que cero salvo en ajustes), construye las entidades, invoca los DAO y devuelve DTO. Sus dependencias se inyectan por constructor, lo que permite probarla con DAO sustitutos sin base de datos. La Tabla 5 enumera sus operaciones públicas.

Tabla 5

Operaciones públicas de la fachada

Método	Regla o coordinación	DAO
listarInsumos() / listarProductos()	Agrega el nombre de categoría y la alerta de reabastecimiento.	InsumoDAO / ProductoDAO, CategoriaDAO
obtenerResumen()	Totales, alertas y últimos cinco movimientos en una sola llamada.	InsumoDAO, ProductoDAO, KardexDAO
registrarInsumo(InsumoDTO)	Textos obligatorios y longitudes; decimales de 0 a 99 999 999.99 con dos decimales; categoría válida.	InsumoDAO
guardarProducto(ProductoDTO)	Alta con stock cero o edición de datos comerciales; producto inexistente → error de negocio.	ProductoDAO
registrarMovimiento(SolicitudMovimientoDTO)	Clase y tipo desde enumeraciones cerradas; motivo obligatorio; delega la transacción.	KardexDAO
ajustarStockInsumo(id, stock)	Registra el conteo físico como AJUSTE con motivo estándar.	KardexDAO
consultarKardex(tipo, desde, hasta)	Filtro de tipo opcional; rechaza una fecha inicial posterior a la final.	KardexDAO

La fachada también centraliza la traducción de excepciones: los controladores solo capturan AplicacionException, cuyo mensaje siempre es apto para el usuario, y el detalle técnico queda en el registro del servidor (Tabla 6).

Tabla 6

Traducción de errores en la fachada

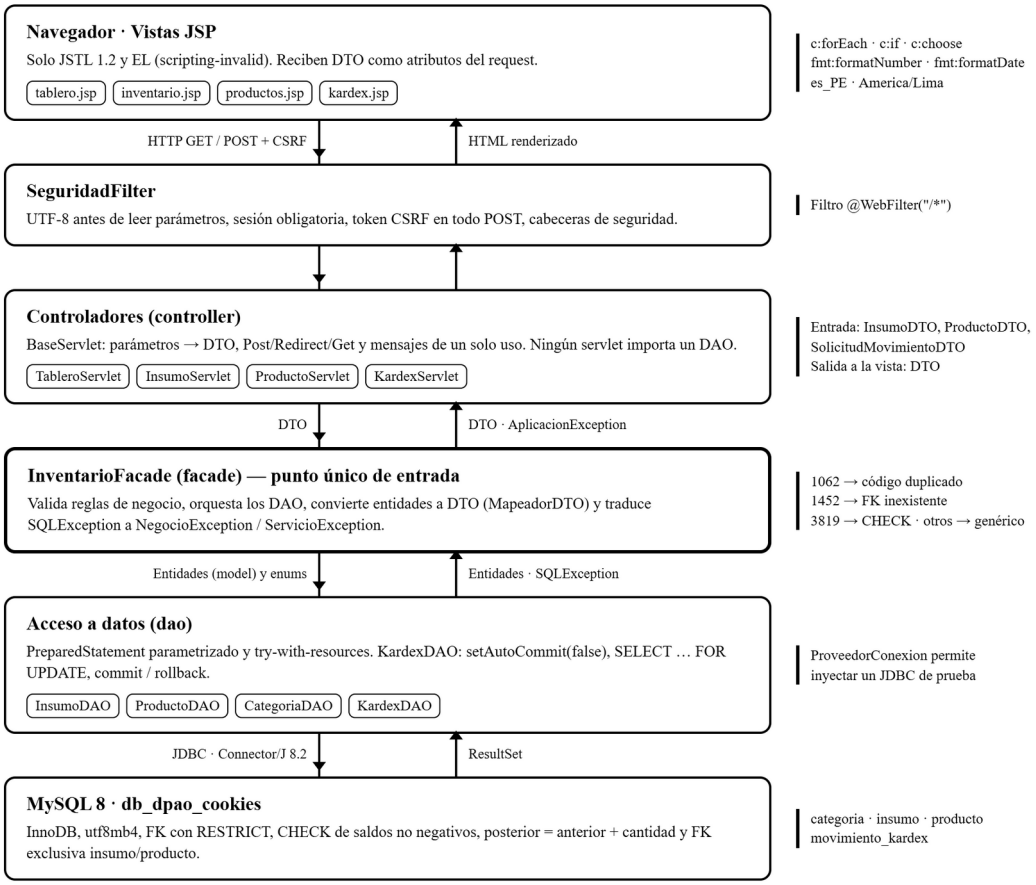
Origen	Excepción resultante	Mensaje mostrado
MySQL 1062 (clave duplicada)	NegocioException	El código ya existe.
MySQL 1452 (FK inexistente)	NegocioException	La categoría o el ítem seleccionado no existe.
MySQL 3819 (CHECK violado)	NegocioException	Los datos no cumplen las reglas del inventario.
Saldo resultante negativo	StockInsuficienteException	Stock insuficiente: disponible X, solicitado Y.
Cualquier otro SQLException	ServicioException (se registra con la causa)	No se pudo guardar la operación o consultar la información. Intente nuevamente.

Diagrama conceptual de capas

La Figura 2 resume el recorrido de una solicitud: la vista envía el formulario, el filtro aplica seguridad, el controlador construye el DTO, la fachada valida y coordina, el DAO ejecuta SQL y la respuesta regresa como DTO hasta la JSP.

Figura 2

Arquitectura en capas: Servlet → InventarioFacade → DAO → MySQL



Nota. Elaboración propia a partir del código de la rama apf2.

Módulo transaccional de Kardex

Cada movimiento modifica dos tablas: el stock del ítem (insumo o producto) y la fila del historial en movimiento_kardex. Ambas escrituras deben confirmarse juntas o no confirmarse; de lo contrario, el saldo y el historial divergen.

Atomicidad transaccional JDBC

JDBC confirma cada sentencia automáticamente salvo que se desactive ese modo; al hacerlo, las sentencias se agrupan en una transacción que se confirma con commit() o se deshace con rollback() (Oracle, s.f.-b). La Tabla 7 describe la secuencia implementada y la Figura 3, su código.

Tabla 7

Secuencia transaccional de KardexDAO.registrar

Paso	Operación	Efecto
1. Iniciar	Connection.setAutoCommit(false)	Las sentencias siguientes forman una sola unidad.
2. Bloquear	SELECT stock_actual, codigo, nombre … FOR UPDATE	Obtiene el saldo anterior y bloquea la fila del ítem.
3. Calcular	TipoMovimiento.variacion(valor, anterior)	Variación con signo y saldo posterior.
4. Validar	posterior < 0 → StockInsuficienteException	Ninguna salida, venta, merma o ajuste deja saldo negativo.
5. Actualizar	UPDATE <tabla> SET stock_actual = ?	Escribe el nuevo saldo.
6. Registrar	INSERT INTO movimiento_kardex (…)	Guarda cantidad, stock_anterior, stock_posterior y motivo.
7. Confirmar	commit()	Hace visibles ambos cambios y libera el bloqueo.
Fallo	rollback() en el bloque catch	Cualquier SQLException o RuntimeException deshace todo; la excepción original se propaga.

Figura 3

Método registrar de KardexDAO: confirmación o reversión de toda la operación

```
try (Connection connection = conexiones.obtener()) {
    connection.setAutoCommit(false);
    try {
        MovimientoKardex movimiento = aplicar(connection, clase, idItem, tipo, valor,
observacion);
        connection.commit();
        return movimiento;
    } catch (SQLException | RuntimeException exception) {
        deshacer(connection, exception);   // rollback()
        throw exception;
    }
}
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/dao/KardexDAO.java. Si el propio rollback falla, ese error se adjunta como excepción suprimida.

Bloqueo pesimista y trazabilidad de saldos

SELECT … FOR UPDATE es una lectura con bloqueo: InnoDB bloquea las filas leídas como si se fueran a actualizar, y otras transacciones que intenten bloquearlas esperan hasta que la primera termine (Oracle, s.f.-a). Así, si dos usuarios registran movimientos sobre el mismo ítem, el segundo lee el saldo ya actualizado y se evita la actualización perdida. La tabla y la columna se toman de la enumeración ClaseItem, nunca de un parámetro del usuario, y los valores viajan con PreparedStatement.

La trazabilidad se garantiza en dos niveles: la aplicación guarda stock_anterior y stock_posterior en la misma transacción, y el esquema lo refuerza con las restricciones CHECK de la Tabla 8, que MySQL aplica desde la versión 8.0.16.

Tabla 8

Restricciones de movimiento_kardex que respaldan la trazabilidad

Restricción	**Regla**
CHECK (stock_posterior = stock_anterior + cantidad)	El saldo final siempre se deriva del inicial y del movimiento.
CHECK (stock_anterior >= 0) y CHECK (stock_posterior >= 0)	Ningún saldo registrado es negativo.
CHECK de signo por tipo	SALIDA, VENTA y MERMA negativas; ENTRADA positiva; AJUSTE con cualquier signo distinto de cero.
CHECK ((id_insumo IS NOT NULL) <> (id_producto IS NOT NULL))	Cada movimiento referencia exactamente un ítem mediante FK.
CHECK (stock_actual >= 0) en insumo y producto	Última defensa si una escritura evitara la aplicación.

La Tabla 9 reproduce la cadena de saldos registrada en la demostración (Figura 9): cada stock_anterior coincide con el stock_posterior del movimiento previo del mismo ítem.

Tabla 9

Cadena de saldos observada en la base de demostración

Ítem	Tipo	Cantidad	Anterior	Posterior	Motivo
HAR-001 Harina de trigo	ENTRADA	+10.00	12.00	22.00	Compra a proveedor
HAR-001 Harina de trigo	SALIDA	−4.50	22.00	17.50	Producción del lote
GAL-001 Galleta de chocolate	ENTRADA	+24.00	18.00	42.00	Horneado terminado
GAL-001 Galleta de chocolate	VENTA	−6.00	42.00	36.00	Venta en stand
GAL-003 Galleta de vainilla	MERMA	−1.00	16.00	15.00	Galletas rotas
MAN-001 Mantequilla	AJUSTE	−0.50	6.00	5.50	Conteo físico
GAL-002 Galleta de avena	VENTA 50.00	Rechazada	12.00	12.00	Sin movimiento (Figura 10)

Nota. Datos ficticios registrados mediante la aplicación sobre la base dpao_demo.

Saldo no negativo y ajustes físicos

La regla de saldo no negativo se evalúa dentro de la transacción, después de bloquear la fila, para que la decisión use el saldo vigente. Si falla, StockInsuficienteException (subclase de NegocioException) provoca el rollback y el usuario recibe el disponible y lo solicitado; no se escribe ninguna fila (Figura 4).

Figura 4

Método aplicar de KardexDAO: cálculo del saldo y validación

```
BigDecimal anterior = movimiento.getStockAnterior();
BigDecimal variacion = tipo.variacion(valor, anterior);
BigDecimal posterior = anterior.add(variacion);
if (posterior.signum() < 0) {
   throw new StockInsuficienteException(anterior, valor);
}
if (variacion.signum() == 0) {
   return null;   // ajuste sin diferencia: no se crea un movimiento vacío
}
```

Nota. Fragmento de src/main/java/pe/com/dpaocookies/dao/KardexDAO.java.

El ajuste por conteo responde a la operación real del negocio: tras un inventario físico se informa la cantidad contada y el sistema registra la diferencia, de modo que el historial explica cualquier discrepancia sin editar saldos manualmente. La Tabla 10 resume la semántica de cada tipo.

Tabla 10

Semántica de TipoMovimiento

Tipo	Valor recibido	Variación registrada
ENTRADA	Cantidad positiva	+cantidad
SALIDA, VENTA, MERMA	Cantidad positiva	−cantidad
AJUSTE	Stock físico contado (puede ser 0)	Contado − anterior; si es cero, no se registra movimiento

Capa de presentación con JSTL 1.2 y EL

Erradicación total de scriptlets

El único scriptlet del APF1 estaba en index.jsp y se reemplazó por <c:redirect>. Para que la regla no dependa de la disciplina del desarrollador, web.xml declara el elemento scripting-invalid, definido por la especificación JSP para grupos de páginas (Oracle, 2013), con valor true para todas las JSP y JSPF: si alguien introduce <% … %> o <%= … %>, el contenedor rechaza la página al compilarla. Las vistas compilaron en Tomcat 9.0.115 con esta restricción activa (Figura 5 y Tabla 11).

Figura 5

Configuración JSP y formato regional en web.xml

```
<context-param>
   <param-name>javax.servlet.jsp.jstl.fmt.locale</param-name>
   <param-value>es_PE</param-value>
</context-param>
<context-param>
   <param-name>javax.servlet.jsp.jstl.fmt.timeZone</param-name>
   <param-value>America/Lima</param-value>
</context-param>
<jsp-property-group>
   <url-pattern>*.jsp</url-pattern>
   <url-pattern>*.jspf</url-pattern>
   <page-encoding>UTF-8</page-encoding>
   <scripting-invalid>true</scripting-invalid>
</jsp-property-group>
```

Nota. Fragmento de src/main/webapp/WEB-INF/web.xml.

Tabla 11

index.jsp antes y después

APF1 (scriptlet)	APF2 (JSTL)
<% response.sendRedirect(request.getContextPath() + "/acceso"); %>	<c:redirect url="/acceso"/>

JSTL, EL y formateo regional

Las listas se recorren con c:forEach, las condiciones con c:if y c:choose, y todo texto proveniente del usuario se imprime con c:out, que escapa el HTML y previene XSS (Sun Microsystems, 2006). Los importes y cantidades usan fmt:formatNumber; el Kardex muestra el signo explícito con el patrón +#,##0.00;-#,##0.00. Las fechas usan fmt:formatDate, que con la zona America/Lima presenta la hora local (UTC−5) independientemente de la zona del servidor. La Tabla 12 detalla el uso por vista y la Figura 6 muestra una fila del historial.

Tabla 12

Uso de JSTL y EL por vista

Vista	Etiquetas y expresiones principales
tablero.jsp	c:if (resumen disponible), c:choose (sin alertas), c:forEach sobre resumen.alertas y resumen.ultimosMovimientos, fmt:formatNumber, fmt:formatDate
inventario.jsp	c:forEach de categorías e insumos, c:choose para [REABASTECER] u [ÓPTIMO], fn:length, fn:escapeXml, fmt:formatNumber
productos.jsp	c:forEach del catálogo, selección de categoría con el operador ternario de EL, c:out en atributos value
kardex.jsp	optgroup de insumos y productos, c:forEach de tiposMovimiento (enum), param.tipo para conservar filtros, fmt:formatDate dd/MM/yyyy HH:mm:ss
nav.jspf	aria-current según el atributo seccion que define BaseServlet

Figura 6

Fila del historial en kardex.jsp con EL y fmt

```
<c:forEach var="m" items="${movimientos}">
  <tr>
    <td><fmt:formatDate value="${m.fechaRegistro}" pattern="dd/MM/yyyy
HH:mm:ss"/></td>
    <td><c:out value="${m.nombreItem}"/> (<c:out value="${m.codigoItem}"/>)</td>
    <td><span class="badge ${m.entrada ? 'badge-success' : 'badge-danger'}">$
{m.tipoMovimiento}</span></td>
    <td class="num"><fmt:formatNumber value="${m.cantidad}" pattern="+#,##0.00;-
#,##0.00"/></td>
    <td class="num"><fmt:formatNumber value="${m.stockAnterior}"
minFractionDigits="2" maxFractionDigits="2"/></td>
    <td class="num"><fmt:formatNumber value="${m.stockPosterior}"
minFractionDigits="2" maxFractionDigits="2"/></td>
  </tr>
</c:forEach>
```

Nota. Fragmento de src/main/webapp/WEB-INF/views/kardex.jsp.

Además se mejoró la usabilidad sin JavaScript: el Kardex selecciona el ítem por código y nombre (con su stock actual) en lugar de pedir un identificador numérico, el inventario elige la categoría de una lista y la navegación marca la sección activa.

Verificación, aseguramiento de calidad y pruebas

Las pruebas se ejecutaron en el entorno descrito en la Tabla 13, con una instancia de MySQL aislada y datos ficticios.

Tabla 13

Entorno de verificación

Elemento	Versión observada
JDK	Eclipse Temurin 17.0.20.1+1
Maven	3.9.16 (JUnit Jupiter 5.10.2, Surefire 3.2.5)
Tomcat	9.0.115 (javax.servlet 4.0.1, JSP 2.3.3, JSTL 1.2)
MySQL	Community 8.4.9, instancia aislada 127.0.0.1:3307 con datos ficticios
Connector/J	8.2.0
Sistema	Windows 11, x86_64

Resumen de las 41 pruebas automatizadas

Las 29 pruebas unitarias no requieren base de datos: emplean dobles de prueba construidos sin librerías externas, un JDBC simulado con java.lang.reflect.Proxy (JdbcFalso) y DAO sustitutos que heredan de los reales. Las 12 de integración se habilitan con DPAO_JDBC_TEST=true y solo se ejecutan contra 127.0.0.1:3307/dpao_test; cada corrida crea un esquema dpao_test_<marca> nuevo y lo conserva como evidencia. La Tabla 14 las agrupa por clase y la Figura 7 reproduce el resultado de Maven.

Tabla 14

Pruebas por clase

Clase	Tipo	N.º	Qué demuestra
KardexDAOTransaccionTest	Unitaria (Proxy JDBC)	8	Orden setAutoCommit(false) → FOR UPDATE → UPDATE → INSERT → commit; rollback ante fallo del INSERT con el UPDATE ya ejecutado; stock insuficiente sin escrituras; ajuste sin diferencia; ítem inexistente.
InventarioFacadeTest	Unitaria (DAO sustitutos)	15	Validación previa al DAO, mapeo a DTO, traducción de 1062 y de fallos técnicos, delegación al Kardex y fachada con KardexDAO real sobre JDBC simulado que hace rollback.
InsumoTest	Unitaria	6	Alerta de reabastecimiento, límites decimales y de texto, variación por TipoMovimiento y rechazo de clases o tipos arbitrarios.
JdbcTest	Integración MySQL	6	Conexión real y fallo de conexión, UTF-8, duplicados y FK, CHECK, rollback por trigger y diez entradas concurrentes sin pérdida.
InventarioFacadeIntegracionTest	Integración MySQL	6	Saldos encadenados ENTRADA → VENTA → MERMA → AJUSTE, venta sin stock sin efectos, rollback por trigger a través de la fachada, ajuste sin diferencia, duplicado y ventas concurrentes.

Figura 7

Resultado de mvn -B test contra MySQL 8.4.9

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0 · dao.KardexDAOTransaccionTest
Motor real: MySQL 8.4.9; esquema aislado: dpao_test_1791495894252
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 · facade.InventarioFacadeIntegracionTest
Tests run: 15, Failures: 0, Errors: 0, Skipped: 0 · facade.InventarioFacadeTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 · model.InsumoTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 · model.JdbcTest
Tests run: 41, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Nota. Extracto de artifacts/evidence/apf2-20261008/maven-test.log.

Concurrencia multihilo y rollback forzado

Concurrencia. La prueba ventasConcurrentesNuncaDejanStockNegativo registra una entrada de 5 unidades y lanza 10 ventas de 1 unidad desde un grupo de 4 hilos a través de la fachada. El bloqueo FOR UPDATE serializa las transacciones: exactamente 5 ventas se confirman, 5 reciben StockInsuficienteException, el stock final es 0.00 y el historial contiene 6 movimientos. JdbcTest comprueba además que 10 entradas concurrentes de 0.10 suman 1.00 sin actualizaciones perdidas.

Rollback forzado. Para provocar un fallo real después del UPDATE, la prueba crea temporalmente un trigger BEFORE INSERT ON movimiento_kardex que ejecuta SIGNAL SQLSTATE '45000'. El INSERT falla, KardexDAO ejecuta rollback() y la prueba verifica en MySQL que el stock conserva su valor previo y que no se añadió historial; desde la fachada, el error llega como ServicioException con mensaje genérico. La versión unitaria reproduce el mismo escenario con el JDBC simulado y verifica que se invocó rollback() y nunca commit().

Pruebas HTTP de humo

El script scripts/http_smoke.py (Python 3, biblioteca estándar) se ejecutó contra el WAR desplegado en Tomcat 9 conectado a la base aislada, y se detiene si el servidor no declara la cabecera X-DPAO-Test-Database: isolated. Realizó 55 solicitudes y todas devolvieron el estado esperado (Tabla 15).

Tabla 15

Comprobaciones del flujo HTTP

Aspecto	Comprobación
Seguridad	Redirección a /acceso sin sesión; credenciales inválidas rechazadas; CSRF ausente → 403; WEB-INF no accesible → 404; cierre de sesión por POST.
Inventario	Alta con UTF-8, alerta [REABASTECER], código duplicado, seis entradas inválidas y ajuste registrado en Kardex.
Productos	Alta y edición con mensajes distintos.
Kardex	Entrada con mensaje «stock 0.00 → 3.00», venta, filtros por tipo y fecha, rango invertido, stock insuficiente sin movimiento, AJUSTE con saldo y ajuste sin diferencia.
Presentación	Cinco vistas navegables; PRG consume el mensaje una sola vez; tablero con últimos movimientos.

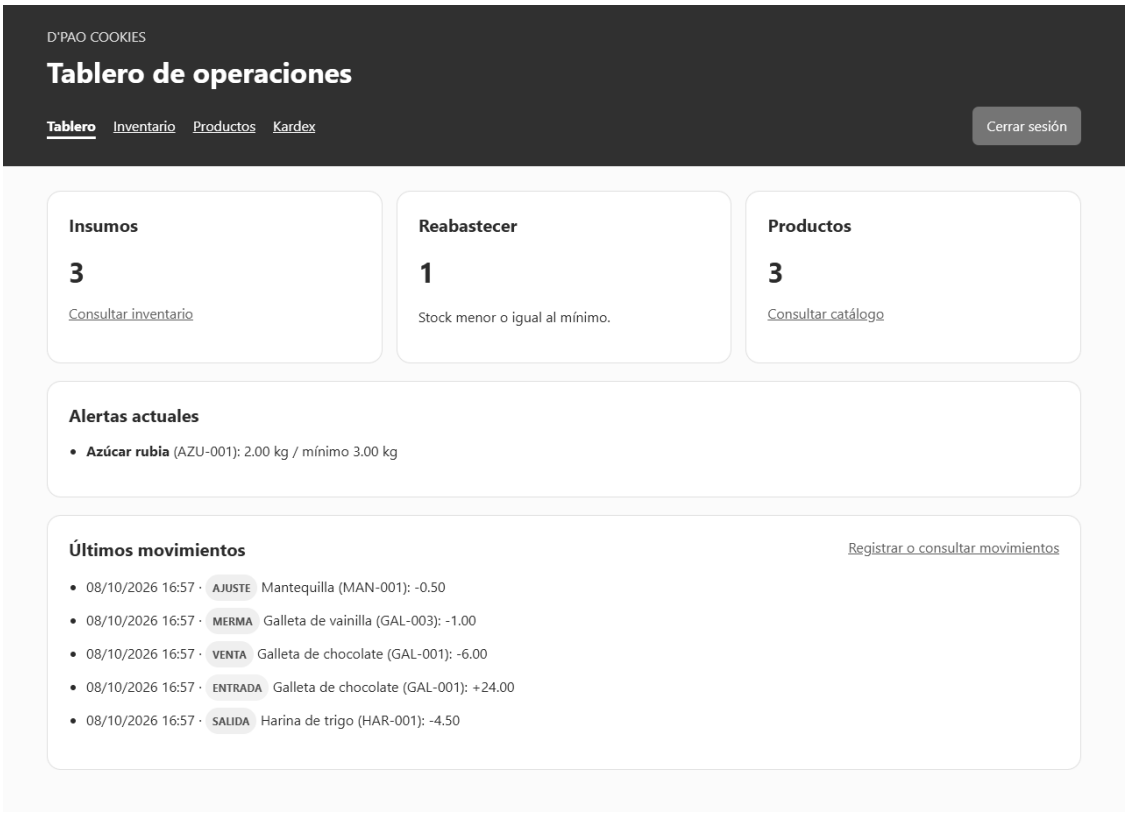
Nota. Los tiempos registrados en el log corresponden a solicitudes locales secuenciales y no constituyen una prueba de carga.

Evidencias del despliegue y ejecución

Las capturas de las Figuras 8 a 12 se tomaron con Microsoft Edge en modo sin interfaz sobre el WAR del commit 3a13408, desplegado en Tomcat 9.0.115 y conectado a una base dpao_demo creada con schema.sql y demo.sql. Los movimientos de la jornada se registraron mediante la propia aplicación; las imágenes se presentan en escala de grises.

Figura 8

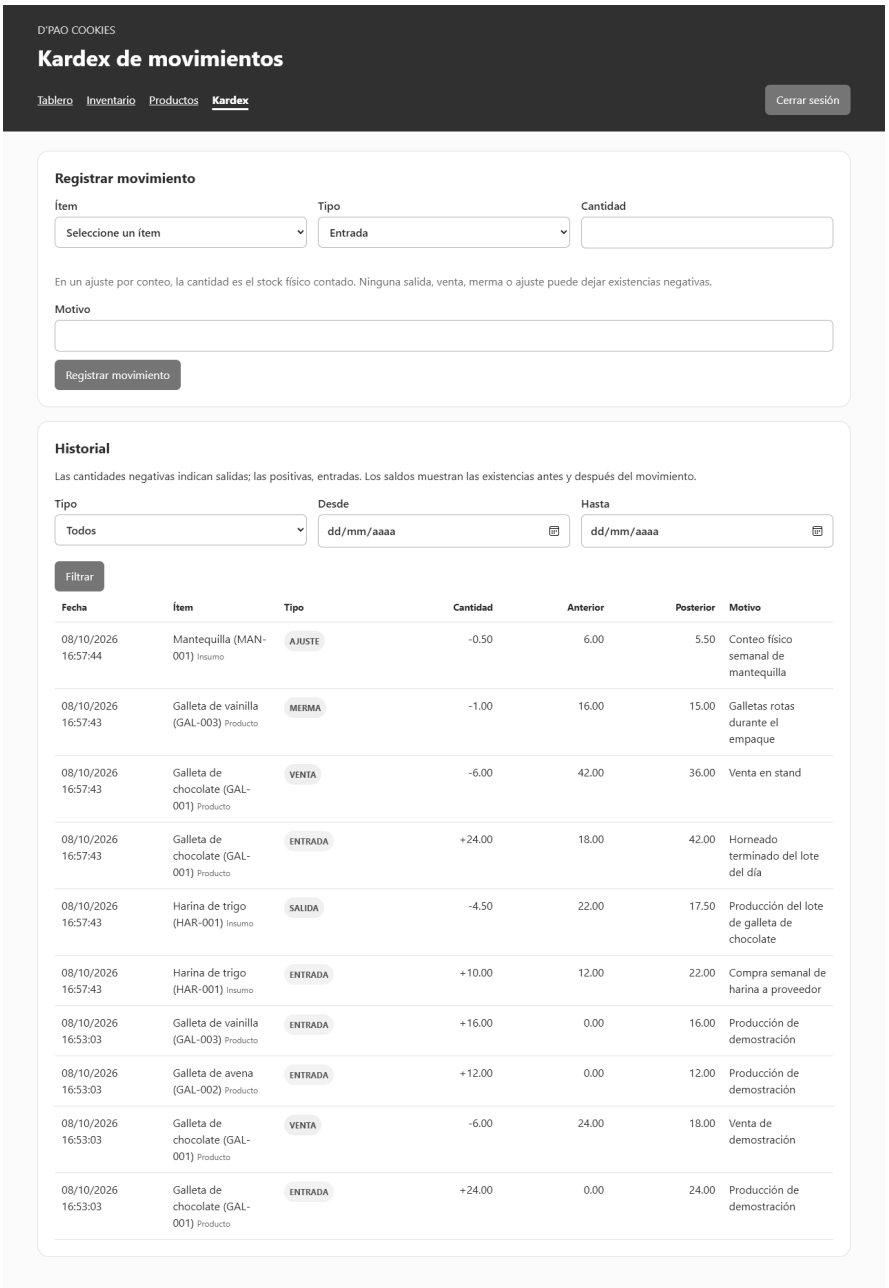
Tablero con indicadores, alerta de reabastecimiento y últimos movimientos



Nota. Archivo de evidencia: artifacts/evidence/apf2-20261008/screenshots/01-tablero.png.

Figura 9

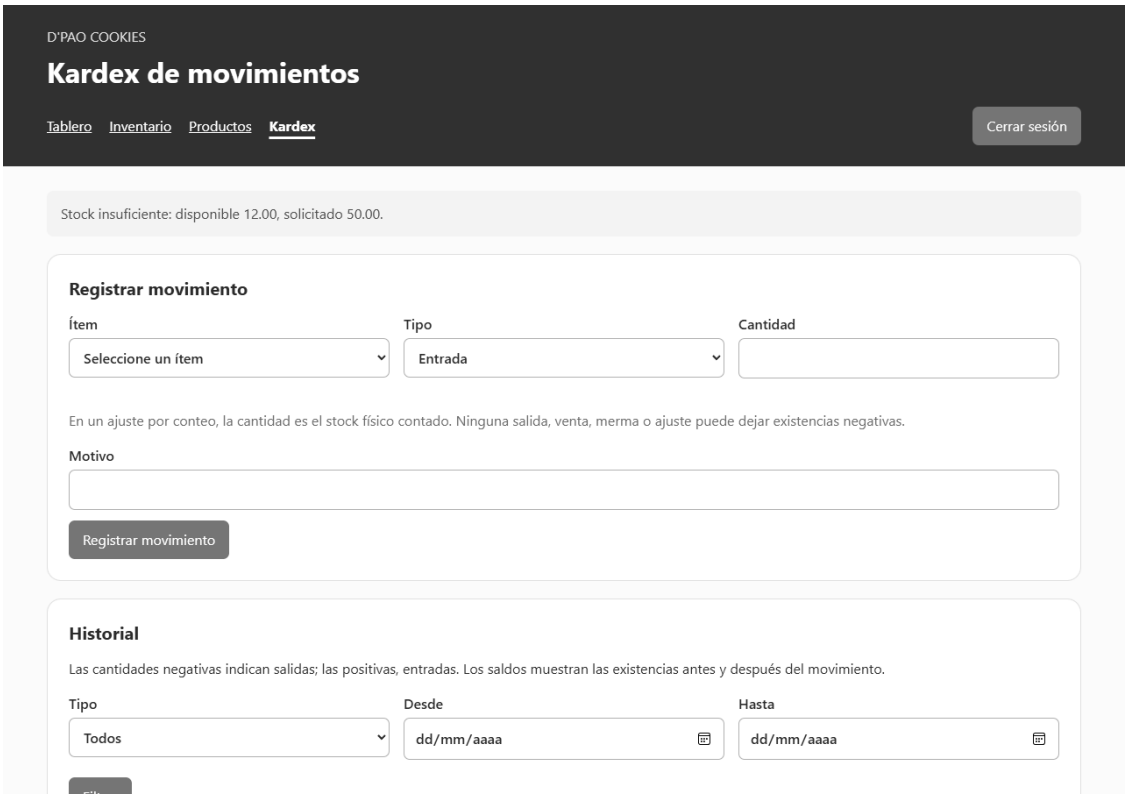
Kardex con saldos anterior y posterior, cantidades con signo y hora de Lima



Nota. Archivo de evidencia: artifacts/evidence/apf2-20261008/screenshots/03-kardex-saldos.png.

Figura 10

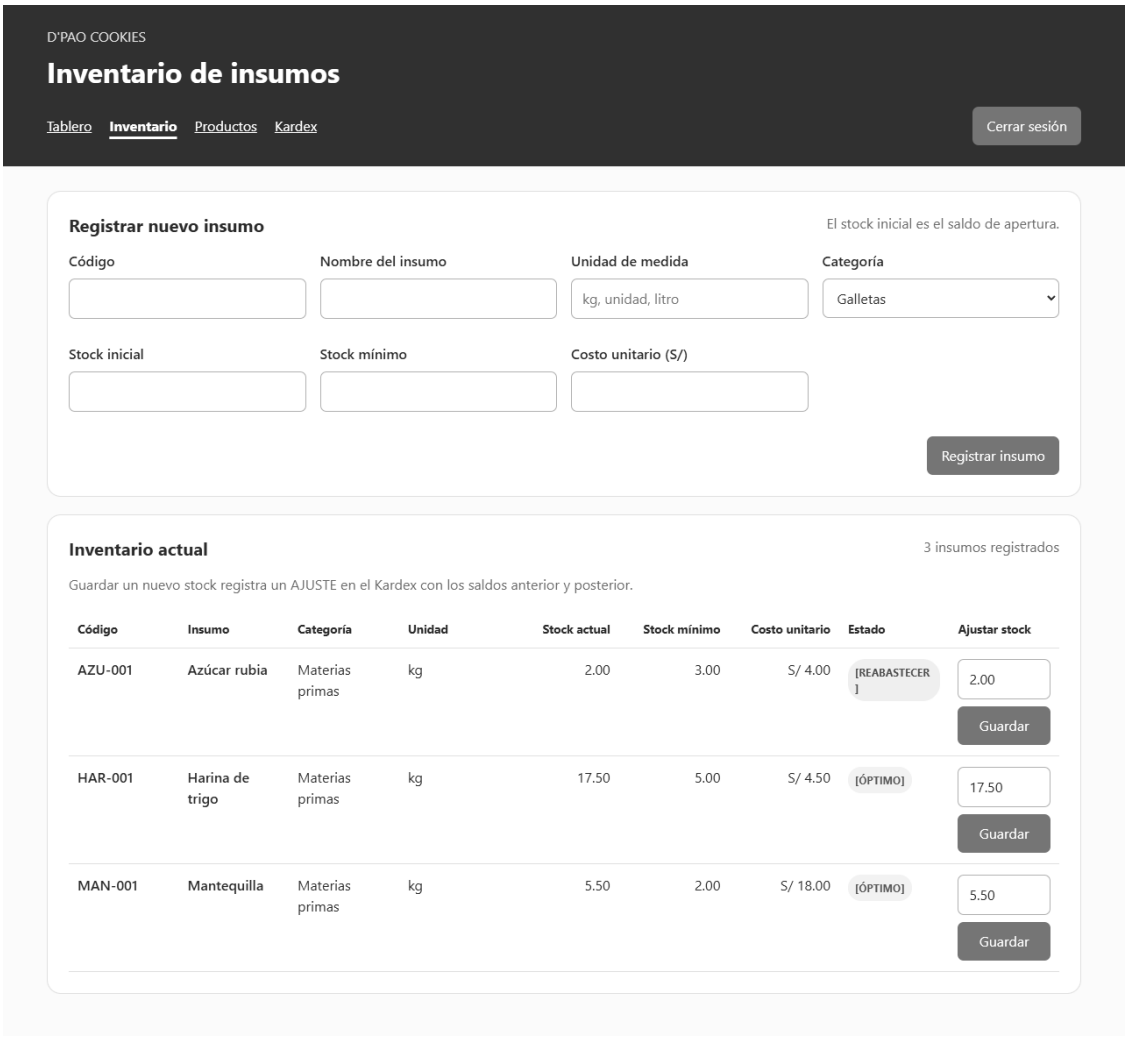
Venta de 50 unidades rechazada por StockInsuficienteException con rollback



Nota. Archivo de evidencia: artifacts/evidence/apf2-20261008/screenshots/04-kardex-stock-insuficiente.png.

Figura 11

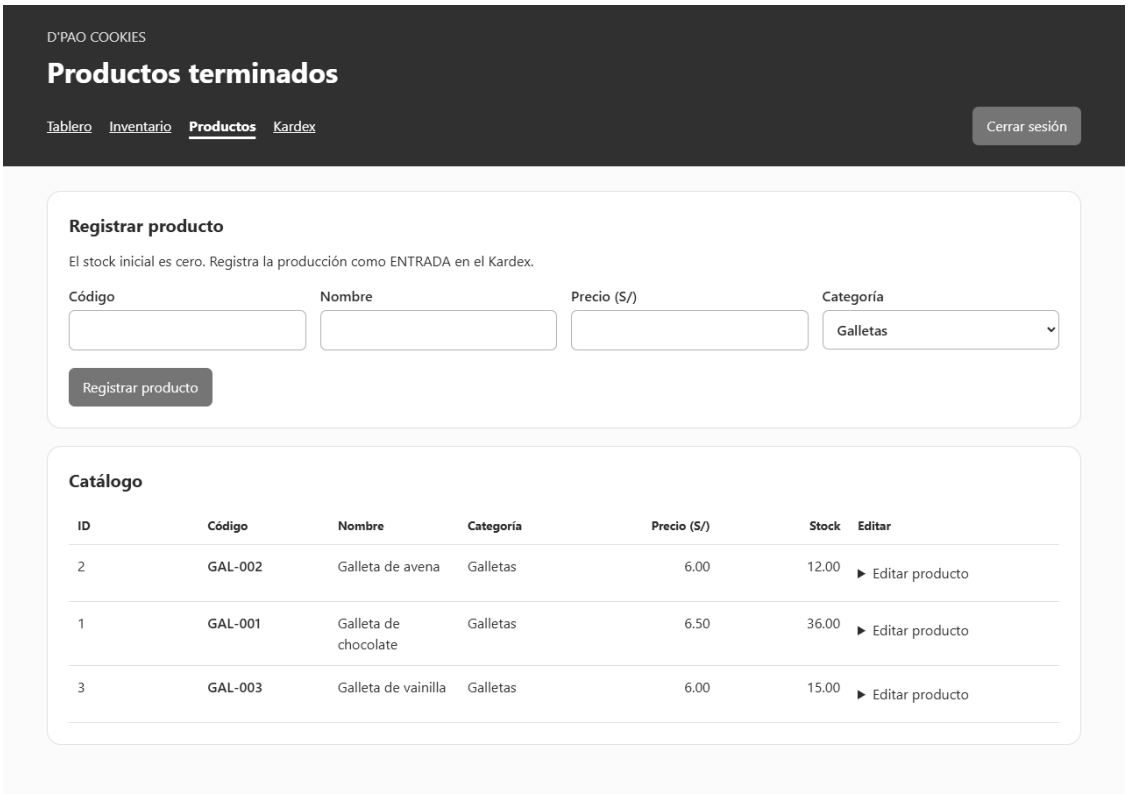
Inventario con categoría, estado de reabastecimiento y ajuste por conteo



Nota. Archivo de evidencia: artifacts/evidence/apf2-20261008/screenshots/02-inventario.png.

Figura 12

Catálogo de productos con stock actualizado por el Kardex



Nota. Archivo de evidencia: artifacts/evidence/apf2-20261008/screenshots/05-productos.png.

Pruebas y control de versiones

La implementación se publicó en la rama apf2 del repositorio. La Figura 13 reproduce el commit y la rama remota que lo contiene, y la Tabla 16 indexa los archivos de evidencia.

Figura 13

Commit de la implementación en la rama apf2

```
commit 3a13408bc42b614fbdf85a7a0bcb5e0ebfa6a3cb
Rama: apf2 (origin/apf2)
feat(apf2): integrar DTO, InventarioFacade y Kardex transaccional
50 files changed, 3452 insertions(+), 412 deletions(-)
Autor: Enrqgonzales <juniorgonzales140@gmail.com>
Contenido en: origin/apf2  (git branch -r --contains)
```

Nota. Extracto de artifacts/evidence/apf2-20261008/git-commit.log.

Tabla 16

Índice de evidencias del APF2

Archivo	Contenido
maven-test.log	mvn -B test con MySQL 8.4.9: 41 pruebas, 0 fallos, 0 omitidas, BUILD SUCCESS (Figura 7).
http-smoke.log	55 solicitudes HTTP y resultado PASS (Tabla 15).
git-commit.log	Commit 3a13408, autor y rama remota que lo contiene (Figura 13).
screenshots/00-acceso.png	Pantalla de acceso con cuenta externa.
screenshots/01-tablero.png	Figura 8.
screenshots/02-inventario.png	Figura 11.
screenshots/03-kardex-saldos.png	Figura 9.
screenshots/04-kardex-stock-insuficiente.png	Figura 10.
screenshots/05-productos.png	Figura 12.

Nota. Rutas relativas a la carpeta artifacts/evidence/apf2-20261008/ del repositorio. Las capturas originales están a color.

Matriz de cumplimiento de la rúbrica

La Tabla 17 relaciona cada criterio técnico del APF2 con la evidencia verificable que lo respalda.

Tabla 17

Matriz de cumplimiento

Criterio	Evidencia verificable	Apartado	Puntaje
Patrones de diseño: MVC + DAO, DTO y Façade	Paquetes controller, dto, facade y dao; ningún servlet importa DAO; MapeadorDTO; traducción de excepciones; InventarioFacadeTest (15).	Arquitectura	5 / 5
Módulo transaccional de Kardex	setAutoCommit(false), FOR UPDATE, commit y rollback, stock_anterior y stock_posterior, StockInsuficienteException, AJUSTE; KardexDAOTransaccionTest (8); rollback por trigger; concurrencia; Figuras 9 y 10.	Kardex	5 / 5
Presentación con JSTL y EL	scripting-invalid en web.xml; c:forEach, c:if, c:choose, fmt:formatNumber, fmt:formatDate; locale es_PE y America/Lima; JSP compiladas en Tomcat.	Presentación	5 / 5
Calidad de software y verificación	41 pruebas aprobadas (29 unitarias y 12 de integración), BUILD SUCCESS, smoke HTTP PASS y commit en GitHub.	Verificación y evidencias	5 / 5
Total			**20 / 20**

Nota. Autoevaluación del estudiante sobre los cuatro bloques del enunciado; la calificación oficial corresponde al docente.

Pendientes de evolución

- Cuentas múltiples con roles y responsable individual registrado en cada movimiento.
- Recetas por producto con consumo automático de insumos al registrar un horneado.
- Paginación del historial del Kardex y exportación de reportes.
- Despliegue público con HTTPS, pool de conexiones JNDI y prueba de carga del requisito de rendimiento.

Referencias

Alur, D., Crupi, J., & Malks, D. (2003). *Core J2EE patterns: Best practices and design strategies* (2.ª ed.). Prentice Hall.

Casas Gonzales, A. E. J. (2026). *dpao-cookies-web* (Rama apf2) [Software]. GitHub. https://github.com/Enrqgonzales/dpao-cookies-web/tree/apf2

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Oracle. (s.f.-a). Locking reads. En *MySQL 8.4 reference manual*. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html

Oracle. (s.f.-b). Using transactions. En *The Java tutorials*. https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html

Oracle. (2013). *JavaServer Pages specification* (Versión 2.3, JSR 245). Java Community Process. https://jcp.org/en/jsr/detail?id=245

Sun Microsystems. (2006). *JavaServer Pages Standard Tag Library* (Versión 1.2, JSR 52). Java Community Process. https://jcp.org/en/jsr/detail?id=52